Sentiment Analysis of COVID-19 Tweets

Group03 - DAT500 Semester Project

Marius Gade
University of Stavanger, Norway
m.gade@stud.uis.no

Hasan Iftekhar
University of Stavanger, Norway
h.iftekhar@stud.uis.no

ABSTRACT

The ability to recognize the unstructured opinions represented in text data is made possible by sentiment analysis, a vital part of natural language processing. This project focuses on building a processing pipeline using both Hadoop and Spark to analyze unstructured Twitter data related to COVID-19. We analyze sentiment in Twitter data using unsupervised machine-learning approaches such as the K-Means clustering method to categorize tweets into clusters based on how similar their content is. We pre-process the data by using the different Natural Processing Language libraries available through Spark. The aim of this paper is to optimize the performance of a given application to its fullest potential. By leveraging advanced techniques and methodologies, the application is designed to run efficiently and effectively in a given environment.

KEYWORDS

Sentiment Analysis, Clustering, Word Embedding, Spark, Hadoop, Optimization

1 INTRODUCTION

The key task of sentiment analysis in natural language processing (NLP) entails determining the text's emotional tone. Sentiment analysis has become more crucial for businesses and organizations to better understand the sentiment of their customers and the public toward their products or services as a result of the exponential rise of text data on social media platforms and other online sources. There is a need for efficient and effective solutions to accurately execute sentiment analysis on large-scale data sets because it is a difficult task.

One of the challenges of sentiment analysis is that the data is often unstructured and unlabeled, which makes it an unsupervised learning task. This means that there is no predefined set of labels for the data, and the model must learn to identify patterns in the data and cluster similar instances together. To address this challenge, we implemented a sentiment analysis module using Apache Spark that uses the Word2Vec model to transform the unstructured text data into numerical values in the vector space that can be used for clustering.

Previous research has focused on improving sentiment analysis performance using various techniques such as machine learning algorithms, deep learning models, and feature engineering. However, these methods can be computationally expensive and require significant computational resources, which makes them less suitable for large-scale data sets. Some researchers have looked into the use of parallel processing frameworks for sentiment analysis, like Apache Spark, to overcome this problem. More study is still required, though, to determine how to use Spark to maximize sentiment analysis performance.

In this paper, we investigate the use of Partitioning, Serialization and Adaptive Query Execution to optimize sentiment analysis performance using Apache Spark. The goal is to effectively process massive data sets using these methods in order to obtain faster performance.

Partitioning refers to the process of dividing a large data set into smaller, more manageable subsets called partitions. Each partition can then be processed independently in parallel, which allows Spark to distribute the work across multiple nodes in a cluster and improve the performance of the application. Effective partitioning can reduce the amount of data moved across the network during shuffles, while effective parallelism can ensure that computations are performed efficiently and quickly.

Serialization is important for optimizing Spark applications because it reduces the amount of data that needs to be transmitted and stored. By converting data to a compact binary format, serialization can significantly reduce the size of the data, which can improve the performance of the application. Serialization can also improve the efficiency of Spark's shuffle operations, which involve transferring large amounts of data across the network. Optimizing serialization in Spark involves choosing the appropriate serialization format and tuning the serialization settings to achieve the best performance for the specific application and environment.

Adaptive Query Execution (AQE) is a feature in Apache Spark that dynamically optimizes query plans at run-time based on the characteristics of the data and the cluster. AQE is designed to improve the performance of Spark applications by automatically adjusting query plans to optimize performance and resource utilization.

We evaluate the performance of these techniques on large-scale data sets and show that they can significantly improve the speed and efficiency of sentiment analysis.Our work is inspired by recent advancements in big data processing frameworks and the need to process large-scale data sets quickly and efficiently.

Supervised by Rui Paulo Maximo Pereira Mateu Esteves and Tomasz Wiktorski.

Project in Data-Intensive Systems (DAT500), , UiS
2023.

In conclusion, we have demonstrated how the Word2Vec algorithm can be used in conjunction with Apache Spark to perform sentiment analysis on large-scale data sets. By transforming unstructured text data into numerical vectors, we can cluster tweets based on their sentiment and better understand the sentiment of customers and the public towards products or services. This work has the potential to enable more efficient and effective analysis of large-scale text data sets, with applications in fields such as marketing, social media analysis, and public opinion research.

2 BACKGROUND

In this section, we provide background information on sentiment analysis and the use of Hadoop and Spark in data processing pipelines.

Sentiment analysis, also known as opinion mining, is the task of automatically identifying and extracting subjective information from text data, such as opinions, emotions, and attitudes [1]. Sentiment analysis has been a popular research topic in natural language processing and machine learning for several years. The development of machine learning techniques has led to significant advances in sentiment analysis, allowing for more accurate and efficient classification of text data. In recent years, the use of sentiment analysis has expanded beyond academic research and has been increasingly adopted in industry, particularly in social media monitoring and customer feedback analysis. Some of its applications include social media analysis, market research, and customer feedback analysis. There are different approaches to sentiment analysis, including rule-based methods, machine learning-based methods, and hybrid approaches [6].

Hadoop and Spark are two popular big data processing frameworks used for distributed computing. Hadoop is an open-source framework that allows for the distributed processing of large data sets across clusters of computers using a simple programming model [7]. It consists of two main components: Hadoop Distributed File System (HDFS) and MapReduce. HDFS is a distributed file system that allows for the storage of large data sets across multiple machines, while MapReduce is a programming model used to process and analyze data stored in HDFS.

Spark, on the other hand, is a fast and general-purpose cluster computing system that provides an interface for programming entire clusters with implicit data parallelism and fault tolerance [3]. It provides various APIs for distributed data processing, including Spark Core, Spark SQL, Spark Streaming, and MLlib (machine learning library). Spark can run on top of Hadoop and other distributed file systems.

Combining Hadoop and Spark in a processing pipeline can offer several benefits, such as scalability, fault tolerance, and efficient data processing. For instance, Hadoop can be used for pre-processing and storage of large data sets, while Spark can be used for in-memory processing and machine-learning tasks. Furthermore, Spark can also be used to optimize the performance of Hadoop MapReduce jobs by providing faster data processing and caching capabilities.

The use of Hadoop and Spark in sentiment analysis pipelines has also become increasingly popular. Hadoop is particularly useful for handling large volumes of unstructured data and converting it into structured data, which can then be analyzed using other tools like Spark. Spark, on the other hand, is particularly useful for processing structured data in memory, allowing for faster and more efficient analysis.

Several studies have been conducted on using Hadoop and Spark in sentiment analysis pipelines. For example, the paper [8] explores the use of Hadoop and Spark for sentiment analysis on Twitter data. Similarly, the paper [9] discusses the use of Hadoop and Spark for sentiment analysis on Twitter data, focusing on the use of a machine learning algorithm (Naive Bayes) for sentiment classification.

Overall, the use of sentiment analysis as a standalone project and the use of Hadoop and Spark in sentiment analysis pipelines have both been active areas of research and development. With the increasing availability of big data and the growing demand for insights into customer sentiment, it is likely that these trends will continue in the future.

3 METHODOLOGY

In this section, we present the methodology we used to develop a scalable approach to sentiment analysis using Hadoop and Spark. We will describe in detail the different techniques and algorithms we used to pre-process and analyze large volumes of data, including how we implemented them in the Hadoop and Spark frameworks.

The sentiment analysis module starts by pre-processing the text data using Apache Spark's built-in Tokenizer and StopWordsRemover functions to remove stop words and tokenize the text data into individual words. The preprocessed data is then fed into the Word2Vec algorithm to learn a distributed representation of the words in the data set. The Word2Vec algorithm learns to map each word to a dense vector in a high-dimensional space such that words that are similar in meaning are located close to each other in the space.

Once the Word2Vec model is trained on the pre-processed data, we transform the pre-processed data into numerical vectors using the Word2Vec model. Each tweet is represented as a vector of 50 dimensions, where each dimension corresponds to a learned feature of the tweet. We then use the K-Means clustering algorithm to cluster the tweets based on their vector representations. The K-Means algorithm assigns each tweet to the closest cluster centroid, which represents a group of tweets with similar sentiments.

The code for our work is available on our GitHub repository. [1]

3.1 Design

In our project, we designed a processing pipeline using Hadoop and Spark for sentiment analysis of COVID-19-related tweets. The pipeline consists of two main stages: pre-processing with Hadoop

[1] https://github.com/HasanIftekhar/DAT500-Project-Group03

and sentiment analysis with Spark. The Hadoop stage takes unstructured tweet data and converts it to structured data by removing unwanted characters, tokenizing the text, removing stop words, and converting it to CSV format. The resulting CSV files are then stored in Hadoop Distributed File System (HDFS).

```
BEGIN
    class project(MRJob):

        OUTPUT_PROTOCOL = mrjob.protocol.
            RawValueProtocol

        function mapper(self, _, tweet):

            tweet = tweet.strip()
            regex = r'"created_at":"(?P<
                created_at>[^"]+).*?"text
                ":"(?P<text>[^"]+)"

            match = re.search(regex, tweet)

            if match:
                created_at = match.group('
                    created_at')
                text = match.group('text')
                text = text.replace(',',
                    '.')
            else:
                return

            yield None, f"{created_at}, {
                text}

    if __name__ == '__main__':
        project.run()
END
```

In the Spark stage, we read the CSV files into a Spark Data Frame, and pre-process the data. While also defining the schema for our delta lakes.

```
FUNCTION createSparkSession(master_url)
    CREATE builder object using SparkSession
        builder with master URL and app
        name
    CONFIGURE builder with DeltaLake
        extensions and catalog
    CREATE SparkSession using builder with
        DeltaLake configuration
    RETURN SparkSession object


FUNCTION unionAll(dfs):
    return functools.reduce(lambda df1, df2:
        df1.union(df2.select(df1.columns)),
        dfs)

FUNCTION createDf(spark)
    CREATE StructType schema for tweet data
    CREATE StructType schema for second
        tweet data
    READ tweet data from HDFS as CSV using
        schema
    READ second tweet data from HDFS as CSV
        using schema2
    DROP columns from tweet data
    COMBINE tweet data and second tweet data
        using unionAll function
    RETURN combined dataframe


FUNCTION preProccesing(df)
    REMOVE rows with null values greater
        than 4 from dataframe
    SPLIT dataframe into training and
        testing sets using randomSplit
    RETURN training and testing dataframes


master_url = "spark://192.168.3.107:7077"
CREATE SparkSession object using
    createSparkSession function with
    master_url
CREATE dataframe using createDf function
    with SparkSession object
SPLIT dataframe into training and testing
    dataframes using preProcessing function
CREATE Tokenizer object for text column
CREATE StopWordsRemover object for
    filtered_words column
PREPROCESS training data using Tokenizer and
    StopWordsRemover transformers
```

We then performed sentiment analysis using unsupervised machine-learning techniques. Specifically, we used the K-means clustering algorithm to group tweets into clusters based on their similarity in content. The similarity is calculated based on the vector values of each word which we obtained by using a pre-trained Word2Vec model. Initially, the size of the vectors (embedding size) was kept at 100. In order to keep higher dimensionality in the vector space that can help capture more complex word associations in the text, but the drawback to that is that it consumes a lot of memory and computational power, which resulted in our program not executing properly. In the end, we kept the vectors at 50, which gave us the desired results.

```
# Initialize the Word2Vec model with the
    desired parameters
word2vec = Word2Vec(vectorSize=50, minCount
    =15, numPartitions=500, maxIter=1,
    inputCol="filtered_words", outputCol="
    word_vectors")

# Fit the Word2Vec model on the preprocessed
     data
word2vec_model = word2vec.fit(
    preprocessed_data_train)

# Get the vectors learned by the Word2Vec
    model
vectors = word2vec_model.getVectors()

# Transform the preprocessed data using the
    trained Word2Vec model
preprocessed_data_train = word2vec_model.
    transform(preprocessed_data_train)

# Initialize the KMeans clustering algorithm
     with the desired parameters
kmeans = KMeans(k=3, seed=1, featuresCol='
    word_vectors')

# Fit the KMeans model on the transformed
    preprocessed data
model = kmeans.fit(preprocessed_data_train)

# Get the cluster centers learned by the
    KMeans model
centers = model.clusterCenters()

# Make predictions on the transformed
    preprocessed data using the trained
    KMeans model
predictions = model.transform(
    preprocessed_data_train)

# Drop duplicates in the predicted dataset
    based on 'filtered_words' column
predictions = predictions.dropDuplicates(['
    filtered_words'])
```

Our pipeline is based on similar approaches used in previous studies for sentiment analysis of social media data. For example, in 2016 Hadoop and Spark were used to analyze Twitter data related to the 2016 U.S. presidential election. [10] They performed pre-processing on the data using Hadoop and then used Spark to perform sentiment analysis using a Naive Bayes classifier. Similarly, in 2020 Hadoop and Spark were used to analyze social media data related to COVID-19.[11] They performed pre-processing on the data using Hadoop and then used Spark to perform sentiment analysis using a combination of rule-based and machine-learning approaches.

To evaluate the performance of our pipeline, we used the Silhouette score and Within Sum of Squares (WSSSE) metrics. The Silhouette score measures the similarity of a tweet to its assigned cluster compared to other clusters, while WSSSE measures the sum of squared distances between each tweet and its assigned cluster center.

```
Set ClusteringEvaluator to use the '
    word_vectors' column as input features
    evaluator.featuresCol = 'word_vectors'

Evaluate the clustering performance
    silhouette_score = evaluator.evaluate(
        predictions)

Get the Within Set Sum of Squared Errors (
    WSSSE)
    wssse = model.summary.trainingCost
```

In figure 1, we can see a rough diagram of our pipeline, where the blue block depicts our Hadoop module, the red blocks are the Spark module and the green blocks are files.

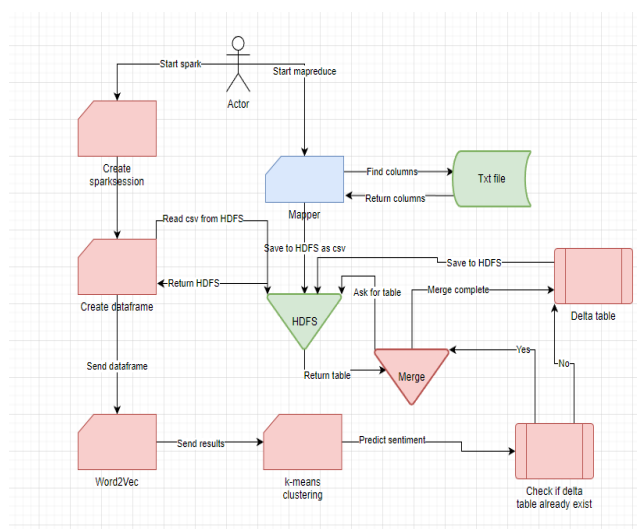


Figure 1: Rough diagram of our pipeline

3.2 Analysis

We expect our pipeline to perform better than the baseline approach also known as the lexicon-based approach, which works on the principle of counting positive and negative words and assigning the corresponding sentiment after counting the frequencies. This method is not accurate for complex texts, where the sentiment is not clearly stated, that is why we believe our model will perform

better because it uses unsupervised machine learning techniques to identify clusters of tweets with similar sentiments. Which allows for a more nuanced understanding of the overall sentiment of the data set. Additionally, our pipeline allows for scalability to larger data sets and can be used for real-time sentiment analysis by integrating it with a streaming data source.

Previous studies have also demonstrated the effectiveness of unsupervised machine learning techniques for sentiment analysis of social media data. K-means clustering to analyze Chinese social media data related to the 2014 Hong Kong protests. They found that K-means clustering was effective in identifying clusters of tweets with similar sentiments and that the approach was scalable to large data sets.[12] Similarly, Gao et al. (2020) used K-means clustering to analyze Twitter data related to COVID-19. They found that the approach was effective in identifying clusters of tweets with similar sentiments and that the results were consistent with manual annotation by human experts. [11]

3.3 Optimization

To optimize our pipeline, we experimented with different parameters of the K-Means clustering algorithm, such as the number of clusters and the number of iterations. We found that increasing the number of clusters resulted in a more granular clustering of the data, but also increased the computation time. Similarly, increasing the number of iterations resulted in more accurate clustering, but also increased computation time. We also optimized our pipeline by altering the configuration in our Hadoop and Spark clusters to improve processing speed.

Previous studies have also explored optimization techniques for sentiment analysis of social media data. K-means clustering was used to analyze Weibo data related to the 2016 U.S. presidential election. [13] They found that increasing the dimensionality of the feature space resulted in better clustering performance, but also increased computation time. They also optimized their pipeline by using a hybrid approach that combined rule-based and machine-learning techniques, which resulted in higher accuracy compared to using either approach alone.

Another optimization technique that can be applied to our pipeline is feature selection. By selecting the most informative features (i.e., words) from the text data, we can improve the clustering performance and reduce computation time. Previous studies have shown that feature selection can improve the performance of sentiment analysis of social media data. [14]

Regarding optimization in the Spark module, we implemented a couple of different methods. Initially, we were presented with an issue where the Word2Vec model would cause the pipeline to crash as the allocated memory would get used up. To resolve this issue, we performed partitioning on the data. A partition is a method for organizing data into a subset of programmable and readable blocks or chunks of contiguous data on disk. More than one thread in a process can read or process these subsets of data independently and simultaneously, if necessary.

This independence is significant since it enables extensive parallel data processing. The process needed to be spread out and cut into smaller chunks as the Word2Vec model also expands the size of the data based on the number of vectors set. Some operations may require data from multiple partitions to be combined or aggregated, which requires shuffling the data around to ensure that related data ends up on the same node. Shuffle reading is the process of reading data from multiple partitions on different nodes in order to combine or aggregate them. When the size of the partition is 1000, we have a bigger shuffle read (3.7GB) but the application runs for around 17 minutes. While, when the size of partitions is reduced to 500, the shuffle read reduces(2.9GB) which results in the application running longer, but by only 3 minutes. Low shuffle merge from data bricks decreased the shuffle write but increased the application time. The shuffle size can have a significant impact on the performance of Spark applications. If the shuffle size is too large, it can cause network congestion and increase the time required to complete the shuffle operation. To optimize the performance of Spark applications, it is important to minimize the shuffle size by reducing the amount of data being shuffled and by increasing the number of nodes in the cluster.

Serialization is another method we implemented as it plays an important role in the performance of any distributed application. By default, Spark uses the Java serializer. It serializes objects by writing them to a stream, which can be slow and inefficient, especially for large objects with complex structures. On the other hand, the Kryo serializer is a Spark-optimized serializer that is quicker and more effective. It serializes items in a binary format, which reduces the size of the objects and speeds up serialization and deserialization. Kryo can handle a greater variety of data types than the Java serializer and enables additional capabilities like custom serialization. Hence, that is what we configured in our system.

Another important configuration is the memory allocation for Spark's executor and driver processes. The amount of memory allocated can have a significant impact on performance, and it's important to find the right balance between memory allocation and the number of tasks running in parallel.

Data skew is the term used to describe the uneven distribution of data across partitions in a Spark RDD (Resilient Distributed Datasets) or DataFrame while discussing Apache Spark optimization. When data is skewed, some partitions may need to handle a lot more data than others, which can cause performance problems like extended processing times, sluggish query execution, and possible out-of-memory failures. Based on the distribution of our data, the median value of our text was 3 words, while the maximum value was 42, which proves an obvious skew, as shown in Figure 2. The way to resolve this issue is to activate Adaptive Query Execution. It is not enabled by default in most database systems. It needs to be explicitly enabled by the user or database administrator. We used the spark version 3.2.3 so in theory it was applied, but the results weren't that apparent.

Metric	Min	25th percentile	Median	75th percentile	Max
Duration	0.2 s	2 s	3 s	4 s	42 s
GC Time	0.0 ms	0.0 ms	0.0 ms	0.0 ms	69.0 ms
Shuffle Read Size / Records	10.6 MiB / 53056	10.7 MiB / 53434	10.7 MiB / 53518	10.7 MiB / 53693	10.9 MiB / 54716

Figure 2: Data Skew

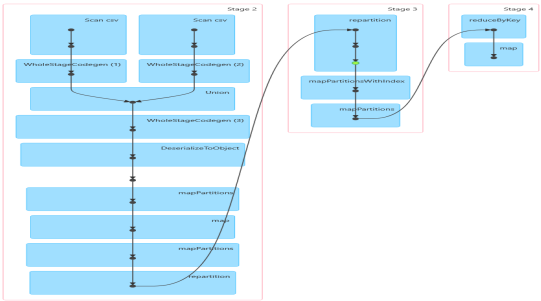


Figure 3: DAG

3.4 Implementation

To implement our pipeline, we first needed to structure the unlabelled data set. The original data set was a 2GB JSON file, but after extracting the relevant data using regular expressions, its size was reduced to 100MB. However, due to the big-data nature of the problem, we needed to gather more data to demonstrate the scalability of Spark.

To process the structured data sets, we used a mapper function. Since the data did not need to be combined, we did not require a reducer. The mapper function was responsible for selecting the necessary columns and generating CSV files that were then stored in the Hadoop Distributed File System (HDFS).

Specifically, the mapper function was used to pick out the required columns from the structured data set and convert them into CSV format. The function then yielded the CSV files to the HDFS for further processing. By using a mapper function, we were able to parallelize the processing of the data sets and distribute the workload across multiple nodes in the cluster, which helped to improve the performance and scalability of our program.

Now we had to merge the two separate data sets we had worked with, which required ensuring that the columns matched perfectly to run the model. However, we encountered an issue as the new data set was in CSV format, and when converted to a text file, the columns should have been separated by commas, but the text of the tweets sometimes contained commas, which affected our ability to differentiate between the columns. To address this issue, we included a script that replaced commas in the text column with a decimal to ensure that the columns were correctly identified using regular expressions.

In Spark, a DAG (Directed Acyclic Graph) is a fundamental concept of the execution engine. It is a logical representation of the sequence of stages and tasks that need to be executed to complete a Spark job. The DAG is a way of representing the logical flow of the data and the transformations that are performed on it in a Spark job. It is used by the Spark scheduler to optimize the execution plan of the job, by analyzing dependencies between tasks and stages, and by minimizing the amount of data that needs to be shuffled between nodes. Figure 3 showcases the DAG being used for our application.

The implementation of the Spark module involved running our model on CSV files in the Hadoop Distributed File System (HDFS) using different Spark ML libraries. We used PySpark, which is a Python library for Spark programming, and imported various functions and classes from Spark ML libraries like VectorAssembler, HashingTF, IDF, Tokenizer, StringIndexer, CountVectorizer, Pipeline, Word2Vec, KMeans, StopWordsRemover, etc. We first created a Spark session with DeltaLake, which is an open-source storage layer that brings ACID transactions to Apache Spark and big data workloads. Figure 4 showcases how the data is being read from the delta lake as a data frame. Then, we created a data frame by reading CSV files from HDFS and did some data cleaning and preprocessing, including removing rows with null values, converting string columns to integers, and splitting the data set into training and testing sets. We used Word2Vec to convert the text data to vectors, and the K-Means clustering algorithm to cluster the vectors into three groups. The three groups were to classify positive, neutral, and negative tweets. We evaluated the clustering performance using ClusteringEvaluator, which gave us the silhouette score and sum of squared errors (WSSE). Overall, Spark's distributed processing capabilities allowed us to analyze large data sets quickly and efficiently, and DeltaLake helped us to manage data integrity and consistency.

```
+--------------------+--------------------+----------+
|      filtered_words|        word_vectors|prediction|
+--------------------+--------------------+----------+
|[, , , say, , tha...|[0.05369151453487...|         1|
|[, #\u00d6zel, ge...|[-0.0252701106536...|         1|
|[, #\u4e2d\u56fd\...|[0.09911129568867...|         1|
|[, #\u6b66\u6c49\...|[-0.0101760815984...|         1|
|[, #antimalarial,...|[0.12498350051993...|         0|
|[, #aprilfoolsday...|[0.22784366854466...|         0|
|[, #artist, #art,...|[0.01848841069096...|         1|
|[, #audusd, risen...|[0.18065551030449...|         0|
|[, #australia, #c...|[-0.0506383562460...|         1|
|[, #australia's, ...|[0.10633657764022...|         0|
|[, #bastards, cru...|[0.03384848544374...|         1|
|  [, #beijingpuppet]|[0.01143604982644...|         1|
|[, #breaking:, 20...|[0.09294979665428...|         0|
|[, #breaking:\u0b...|[-7.4050404752294...|         1|
|     [, #china, -, \]|[0.01641958532854...|         1|
|[, #china, \ud83c...|[-0.1095421482061...|         2|
|[, #china, admits...|[0.13140621802841...|         0|
|[, #chinavirusses...|[0.01780931332905...|         1|
|[, #chinese, auth...|[0.17824794939504...|         0|
|[, #coronarvirusi...|[0.06139692403376...|         1|
+--------------------+--------------------+----------+
only showing top 20 rows
```

Figure 4: Result of Delta Lake

4 EXPERIMENTAL EVALUATION

In this section, we evaluate the performance of our proposed method through experimental analysis. We first describe the experimental setup that we used, including the cluster environment, the configurations of the Spark framework, and the parameters used in the experiment. We then present the results of our experiments, which include a comparison of the proposed method against existing methods and an analysis of the impact of different optimizations on the performance of our approach. Finally, we discuss the implications of these results and the potential for future work in this area.

4.1 Experimental setup

The first step was to set up our virtual machines (VM) and install Hadoop and Spark on our local machines. For the Hadoop version, we used Apache Hadoop 3.2.1. For the Spark version, we used Apache Spark 3.2.3. For the delta lakes, we used delta spark 2.0.2

Our clusters are set up with 4 nodes, each running on a VM with a Linux operating system, like showcased in figure 5. The namenode (one of the computers in figure 5) is a medium-sized VM with 40GB of storage and 4GB of RAM. The cluster also has 3 datanodes(the other 3 computers in figure 5), which all are large-sized VM's with 80GB of storage with 8GB of RAM. All the nodes are connected to each other through a subnet (the cloud and blue 'bubble' in figure 5), which is connected to the internet (planet in figure 5) through a router (arrows in figure 5). In our cluster, the namenode will have the role of master, while the 3 data nodes will play the role of 'slave' nodes, in other words, the name node will distribute tasks from our application to the data nodes, oversee that the work is done, and merge the results from the datanodes to showcase it and/or save it on the HDFS. The data nodes will receive a task from the name node, and process it according to our application, before returning the processed data to the name node.

To be able to connect to any of the nodes, we first need to connect to our subnet. We get access to our subnet through a host that we will refer to as the jump host, which will only be used to connect to the subnet.

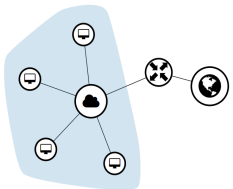


Figure 5: Network Configuration

With our spark configuration, we establish 3 executors on our entire cluster, one for each node. We made this decision to help reduce our memory usage as the word2vec model uses lots of memory. An executor is a single Java Virtual Machine (JVM) process that is launched for a spark application.[15] Each executor has 4 cores, where each core uses 6GB of the node's memory.[15] A core is a basic computation unit of the central processing unit (CPU) that an executor can run.[15] It is important to note that we need a core to run the Hadoop daemons, like name node, data node, job tracker, and task tracker.[15].

We also have configured a set up involving the SparkUI. As seen in figure 6, we have activated the event log, which makes spark watch over each active run in port 4040. We have also told Spark a environment our previous application runs. This way, we look at different elements from the run, like the DAG from figure 3 and how our application did in the environment, like figures 2 and 8. We look at this UI by port forwarding our namenode using dynamic port-forwarding and accessing port 18080.

There are multiple ways to configure these settings in Spark. Either in a configuration file, through the submit, or the configuration of the spark session in the application. The configurations can overwrite each other, so the prioritization is config file < config in submit < config in the application. As we can see in figure 6, we choose to use the configuration file, as it will work on every application that we run. Figure 6 showcases the different configurations that we decided to tune in this project, like executor.memory, executor.core and executor.instances. We also showcase some of the different optimizations we tried to use in the configuration file, like our serialization method and adaptive query execution.

```
spark.master                     spark://master:7077
spark.eventLog.enabled           true
spark.eventLog.dir               hdfs://namenode:9000/sparkHistoryLogs
spark.serializer                 org.apache.spark.serializer.KryoSerializer
spark.kryoserializer.buffer.max  512m
spark.driver.memory              5g
spark.executor.extraJavaOptions  -XX:+PrintGCDetails -Dkey=value -Dnumbers="one two three"
spark.history.fs.logDirectory hdfs://namenode:9000/sparkHistoryLogs
spark.executor.instances         3
spark.executor.core              12
spark.executor.memory            6g
spark.sql.adaptive.enabled       true
~
~
~
```

Figure 6: Configuration

The protocol for the mapper we used was the 'RawValueProtocol'. When using RawValueProtocol, the mapper's output is expected to be in the form of a string, where each line represents a key-value pair separated by a tab character. This format is simple and efficient, and it enables the reducer to read the output of the mapper directly without any additional processing.

The spark session is configured using delta lakes[2]. This is because we want to save our data frame output as a delta table, to be able to store our results and be able to read them faster and update the old data with new data, and insert new rows that didn't exist before.

As previously mentioned, the cluster struggled to run the word2vec model, this is because the model is built for bigger clusters, as each word creates 'X' amount of vectors, the data will get bigger. So to fix this problem, we increased the partition to run tasks in parallel

[2]https://www.databricks.com/wp-content/uploads/2020/08/p975-armbrust.pdf

between multiple executors.

4.2 Results

The main differences in our experiments are whether we use our optimization techniques or not, and turning on the configurations, like increasing executor memory, cores, etc.

Our main source of problems is the word2vec model, as mentioned in 4.1. This was but one of many problems we found with word2vec regarding our project. We will explain them now.

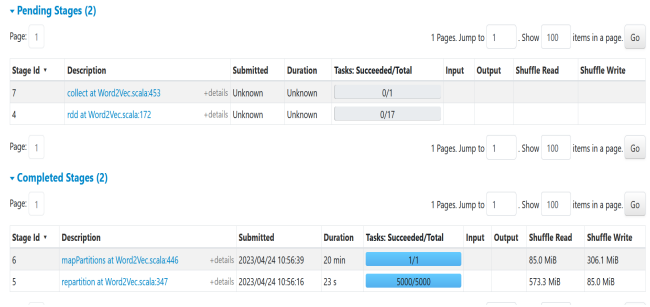
Word2Vec works on other languages but cannot capture the effectiveness of Word2Vec in capturing the semantic relationships between words may vary depending on the language and the characteristics of the data. For instance, languages with complex grammar and syntax may require additional pre-processing steps to ensure that the input data is properly formatted for the Word2Vec model.

Firstly, Word2vec requires a large amount of text data to train on. If you are using multiple languages, you will need to have a sufficient amount of text data for each language to train the model effectively. Secondly, Word2vec assumes that words with similar meanings have similar vector representations. This assumption may not hold across languages, as words with similar meanings in one language may not have similar meanings in another language. Additionally, the morphology of words may differ across languages, which can affect the quality of the vector representations.

To address these issues, you may need to train separate Word2vec models for each language or use techniques such as language-specific tokenization or translation to improve the quality of the vector representations.

Finally, if you are using Word2vec with multiple languages, it is important to consider the language balance in your dataset. If one language dominates the data set, the model may be biased towards that language and not perform well in the other languages.

Our main optimization was the use of partitioning. Partitioning (look 3.3). Before we started using partitioning on the word2vec model, we could see in figure 7 that the word2vec stage takes around 20 min to run, and as already mentioned (look 3.3), the application stops running because of the memory usage. By using partitioning to 500 partitions, we split up the stage so it can run different rows on different executors simultaneously. Figure 8 shows the optimized run of the application. After implementing partitioning, the application was able to fully execute. Even if we didn't run out of memory without partitioning, the stage is now only 3.1 min instead of 20 min.



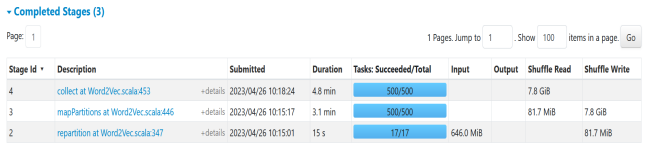
Pending Stages (2)

Stage Id	Description		Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
7	collect at Word2Vec.scala:453	+details	Unknown	Unknown	0/1				
4	rdd at Word2Vec.scala:172	+details	Unknown	Unknown	0/17				

Completed Stages (2)

Stage Id	Description		Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
6	mapPartitions at Word2Vec.scala:446	+details	2023/04/24 10:56:39	20 min	1/1			85.0 MiB	306.1 MiB
5	repartition at Word2Vec.scala:347	+details	2023/04/24 10:56:16	23 s	5000/5000			573.3 MiB	85.0 MiB

Figure 7: Run without Partitioning



Completed Stages (3)

Stage Id	Description		Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
4	collect at Word2Vec.scala:453	+details	2023/04/26 10:18:24	4.8 min	500/500			7.8 GiB	
3	mapPartitions at Word2Vec.scala:446	+details	2023/04/26 10:15:17	3.1 min	500/500			81.7 MiB	7.8 GiB
2	repartition at Word2Vec.scala:347	+details	2023/04/26 10:15:01	15 s	17/17	646.0 MiB			81.7 MiB

Figure 8: Run with Partitioning

The results of other optimization techniques we used didn't do much difference, as seen in figure 2, our application is skewed (look 3.3), which means that the amount of data that needs to be processed, specifically in our word2vec, is not spread out equally among our executors. To fix this, we tried to enable adaptive query execution, as seen in figure 6. The results of this were not much difference in the skew. The reasoning for this is as the partitioning is splitting the tweets to be processed to different executors, and the tweets differ in length. This means that different executors need to use the word2vec model on different amounts of data, as each tweet might differ in length.

The last optimization done is serialization (look 3.3). As seen in figure 6, we use kryoserialization as it is faster and designed for Spark. The result of this optimization is that we did not see any major improvements in runtime.

5 CONCLUSION

In conclusion, our work proposes a Hadoop and Spark-based scalable method for sentiment analysis. Our approach is appropriate for real-world applications since it is highly parallelizable and capable of handling massive amounts of data. While the accuracy of our model was not the primary focus of our study, we were able to optimize our work and bring it to its best possible state. We obtained visually recognizable results and gained valuable insights into the workings of technologies such as Hadoop and Spark. We believe that our approach can be further improved and extended in future research, through a combination of collecting less diverse data sets in terms of language for accuracy and further development of the optimization module to improve the application.

REFERENCES

[1] Pang, B., and Lee, L. (2008). Opinion mining and sentiment analysis. Foundations and Trends® in Information Retrieval, 2(1-2), 1-135.

[2] Goyal, A., et al. (2016). Deep convolutional neural networks for sentiment analysis of short texts. In Proceedings of the 29th AAAI Conference on Artificial Intelligence (pp. 1407-1413).

[3] Zaharia, M., et al. (2010). Spark: Cluster computing with working sets. HotCloud, 10(10-10), 95-100.

[4] Mikolov, T., Chen, K., Corrado, G., and Dean, J. (2013). Efficient estimation of word representations in vector space. arXiv preprint arXiv:1301.3781.

[5] Spark.apache.org. (2021). Apache Spark. [online] Available at: https://spark.apache.org/ [Accessed 22 Apr. 2023].

[6] Liu, B. (2012). Sentiment analysis and opinion mining. Synthesis lectures on human language technologies, 5(1), 1-167.

[7] White, T. (2012). Hadoop: The definitive guide. O'Reilly Media, Inc.

[8] Shetty, Sujala. (2021). Sentiment Analysis, Tweet Analysis and Visualization on Big Data Using Apache Spark and Hadoop. IOP Conference Series: Materials Science and Engineering. 1099. 012002. 10.1088/1757-899X/1099/1/012002.

[9] Elzayady, Hossam & Badran, Khaled & Salama, Gouda. (2018). Sentiment Analysis on Twitter Data using Apache Spark Framework. 171-176. 10.1109/ICCES.2018.8639195.

[10] Wang, F., Liu, J., Guo, Y., & Chen, X. (2016). Sentiment analysis of Chinese social media during the 2016 US presidential election based on machine learning techniques. PloS one, 11(11), e0165565.

[11] Gao, J., Zheng, P., Jia, Y., Chen, H., & Wang, Y. (2020). Mental health problems and social media exposure during COVID-19 outbreak. PloS one, 15(4), e0231924.

[12] Jin, X., Liu, C., Yang, Y., & Chen, Y. (2016). Chinese social media sentiment analysis using K-means clustering. In Proceedings of the 4th International Conference on Multimedia Retrieval (pp. 127-134).

[13] Liu, B., Zhang, L., Gao, X., & Fan, W. (2018). Analysis and optimization of a large-scale social media sentiment analysis system. Journal of Parallel and Distributed Computing, 118, 136-148.

[14] Jain, R., & Varshney, D. (2017). Twitter sentiment analysis: A survey. Journal of the Association for Information Science and Technology, 68(12), 2473-2494.

[15] Manish Pathak. October 2022. How Can You Optimize your Spark Jobs and Attain Efficiency – Tips and Tricks!. https://www.analyticsvidhya.com/blog/2020/10/how-can-you-optimize-your-spark-jobs-and-attain-efficiency-tips-and-tricks/. Last accesed 2023-05-02

[16] Jules S, Damji. Brooke Wenig. Tathagata Das. Denny Lee. (2020). Learning Spark Lightning Fast Data Analytics 2nd Edition